



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



**FACULTAD
DE INGENIERÍA**

Ingeniería del Software II

INFORME RUP

PROYECTO “ZERO E-COMMERCE”

Trabajo Práctico integrador nro 1

Grupo 3:

- Beck Augusto
- Maglione Martín
- Ferrari Paulina

Profesores:

- Spadaro Leandro
- Cortéz Lucía

Año:

- 2024



Ingeniería del Software II

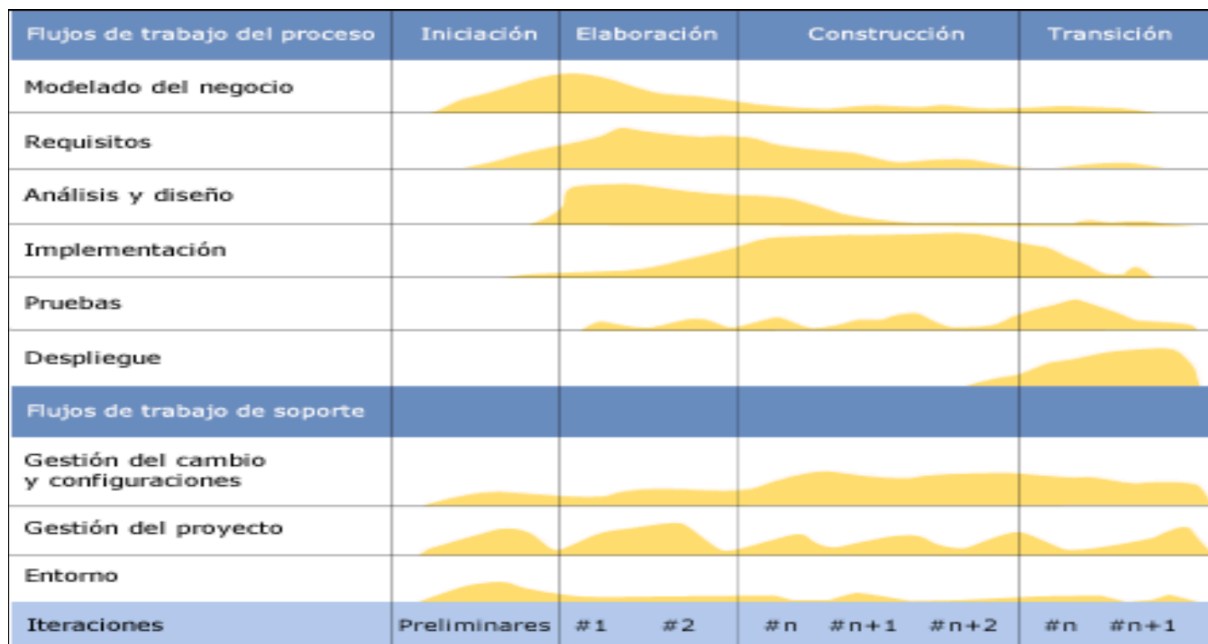
Índice:

1. Metodología
2. Desarrollo de Software
 - Iteración 1
 - Iteración 2
 - Iteración 3
 - Iteración 4
 - Iteración 5
 - Iteración 6
 - Iteración 7
3. Patrones utilizados
4. Propuesta de mejora
5. Tecnologías
6. Requerimientos no funcionales.

Ingeniería del Software II

1. Metodología

“Para la realización del trabajo práctico, el equipo de trabajo utilizará el Proceso Racional Unificado (RUP)”



2. Desarrollo de Software

A) Iniciación

Iteración 1:

- **Modelado del negocio:** lectura exhaustiva del enunciado para la separación del proyecto en distintos paquetes, entendimiento de los requisitos, división inicial de responsabilidades para la creación de las clases entidades, elección de tecnologías.



Ingeniería del Software II

- **Requisitos:** análisis del uml (provisto por el profe) e identificación de los principales requisitos funcionales:
 - Registro de usuarios
 - Carrito persistente a través de sesiones de un usuario
 - Registrar ventas (facturas) virtuales y presenciales
 - Manejo de stock
 - Sistema de mails
- **No se llevaron a cabo las demás actividades**

B) Elaboración

Iteración 2:

- **Modelado del negocio:** lectura de la consigna e investigación del funcionamiento de elementos del mundo real como las facturas y órdenes de compra.
- **Requisitos:** análisis del UML para determinar cómo deben interactuar las entidades y profundizar en ideas de cómo implementar los casos de uso con los que no estaba familiarizado el equipo.
- **Análisis y diseño:** determinación de arquitectura en capas de presentación (vistas y controladores), de dominio, de servicio y de acceso a datos. Elección de plantillas.
- **Implementación:** creación del proyecto, creación de paquetes y clases entidades preliminares. Comienzo de adaptación de las vistas a las pantallas que necesita el proyecto.



Ingeniería del Software II

Iteración 3:

- **Requisitos:** análisis del UML para determinar cómo deben interactuar las entidades.
- **Análisis y diseño:** diagrama de paquetes y DER.
- **Implementación:** avances en todas entidades y vistas. Se implementaron las relaciones entre entidades y se comenzó a incluir thymeleaf en las vistas.

C) Construcción

Iteración 4:

- **Requisitos:** análisis del UML.
- **Análisis y diseño:** comienzo de escenarios y diagramas de caso de uso de los casos Registrar orden de compra, Registrar factura y Actualizar Stock.
- **Implementación:** creación de todos los repositorios y comienzo de servicios y controladores. Implementación de ABM de clases Cliente, Usuario, Dirección, DTOs de paquete zona, Producto, VigenciaPrecio. Se cargaron en la base de datos nacionalidades, países, provincias, departamentos y localidades. Comienzo de AuthController y AdminProduct, implementación zona controller.

Iteración 5:

- **Requisitos:** análisis del UML.



Ingeniería del Software II

- **Implementación:** desarrollo de servicios y controladores. Implementación de ABM de Proveedor, Contacto, Factura, Detalle, Stock, OrdenCompra, DetalleCompra. El equipo decide agrupar los servicios de factura y detalle en VentaService. Implementación de carrito de compra, actualización de stock y compras (registro de facturas). Comienzo de controladores para vistas de operaciones con proveedor (AdminProvider y AdminProviderPurchase). Para este momento el front end está terminado.

Iteración 6:

- **Requisitos:** análisis del UML.
- **Análisis y diseño:** cambios en los escenarios y diagramas. Comienzo de los diagramas de secuencia de diseño.
- **Implementación:** desarrollo de servicios y controladores. Implementación. Controladores para funcionalidades no implementadas hasta el momento: perfiles e historial de compra, email service y newsletter service, integración de sdk de mercado pago.
- **Pruebas:** pruebas unitarias de los casos de uso principales.

D) Transición

Iteración 7:

- **Requisitos:** análisis del UML.



Ingeniería del Software II

- **Análisis y diseño:** cambios en diagramas de secuencia.
- **Implementación:** implementación de reportes y ajustes de código.

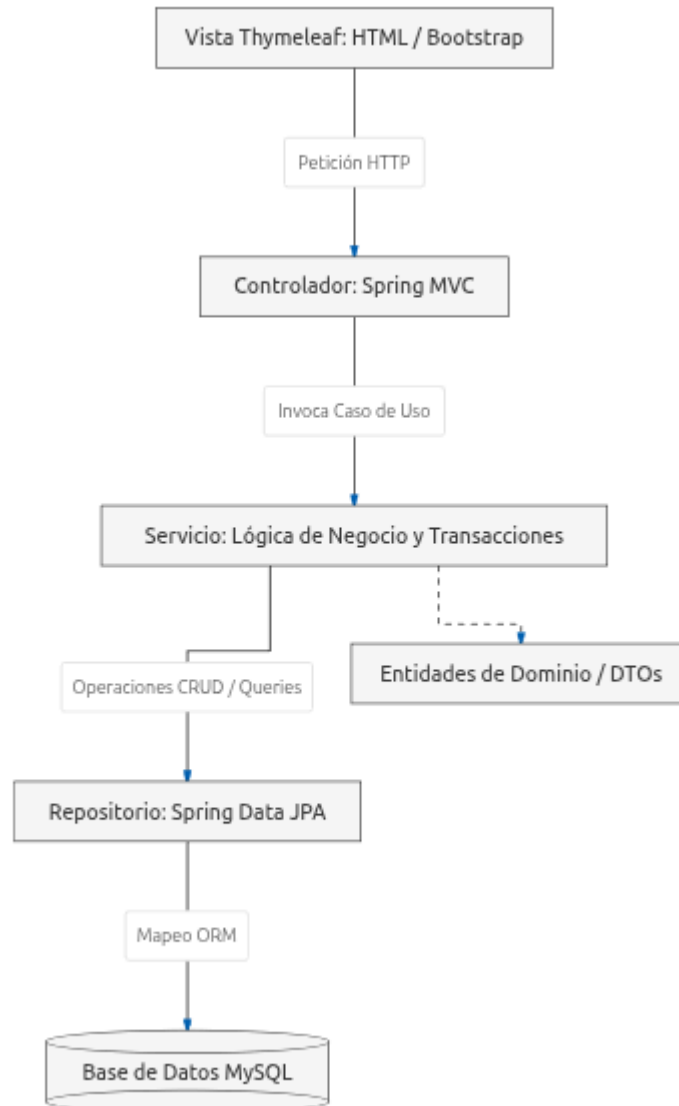
3. Patrones Utilizados

Patrón Arquitectónico en Capas

El proyecto está estructurado en cuatro capas lógicas independientes:

- 1) **Capa de Presentación:** Esta capa abarca los controladores del paquete **controllers** y se encarga de renderizar las vistas del E-commerce Zero.
- 2) **Capa de Lógica de Negocios:** Representada por el paquete **services**. Maneja todas las reglas de negocio como validaciones de precios, existencia de stocks, etc.
- 3) **Capa de Acceso a Datos:** El paquete **repositories** es el representante de esta capa y es donde están todos los repositorios de todas las entidades necesarias para manejar la persistencia de los datos que fluyen por el sistema.
- 4) **Capa de Dominio:** Todas las entidades están agrupadas en el paquete **entidades** y son las clases encargadas de modelar el dominio de los datos que van a aparecer en el proyecto.

Ingeniería del Software II



La aplicación de este patrón es importante para separar las responsabilidades y tener claro quién hace qué a lo largo de todo el ciclo de vida del software. Los controladores solo renderizan vistas, no realizan consultas SQL. Los repositorios solo persisten datos en la base de datos, no validan reglas de negocio. Este patrón también nos aporta mantenibilidad y testeabilidad a la hora de darle soporte.

Ingeniería del Software II

Patrón Experto en Responsabilidad

El principio de Experto asigna la responsabilidad de un cálculo o comportamiento a la clase que posee la información necesaria para realizarlo. Ésto se puede ver reflejado en la clase de StockService dado que posee métodos como calcularStockActual(), disminuirStock(), etc. donde tiene todos los datos necesarios para realizar los cálculos.

Este patrón evita la dispersión de reglas de inventario en controladores o entidades. Al concentrar el cálculo de balance donde residen los datos de auditoría de stock, se garantiza que ninguna venta ni recepción de compra pueda corromper el balance de inventario.

```
@Service
public class StockService {

    @Transactional(readOnly = true)
    public int calcularStockActual(String productoId) {
        Optional<Stock> ultimoStock = stockRepository.findUltimoStockPorProducto(productoId.trim());
        return ultimoStock.map(stock -> Math.max(0, stock.getCantidadActual())).orElse(0);
    }
}
```

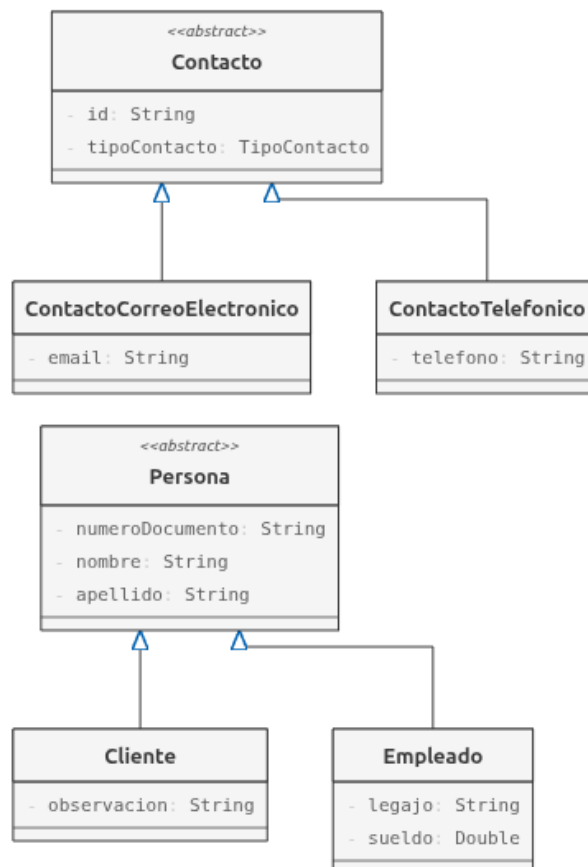
En lugar de que Producto, Factura o los controladores intenten calcular existencias, la responsabilidad es de StockService porque es el único que conoce el historial de movimientos de inventario, egresos por venta e ingresos por compras a proveedores.

Ingeniería del Software II

Polimorfismo

Este patrón se utiliza en las clases de Persona que hereda a Cliente y Empleado, Factura que hereda a FacturaCliente y FacturaProveedor, Contacto que hereda a ContactoCorreoElectrónico y ContacoTelefónico.

Facilita la extensibilidad. Si en el futuro se incorpora un nuevo tipo de persona o un nuevo canal de contacto, no es necesario alterar las relaciones de base de datos ni los servicios que consumen Persona o Contacto.



Ingeniería del Software II

DAO (Data Access Object)

El proyecto utiliza JPA y Hibernate. Todos los repositorios extienden de `JpaRepository<T, ID>` actuando como DAO que provee operaciones de CRUD, paginación y consultas derivadas dejando al programador el desarrollo de métodos correspondientes a consultas JPQL específicas del modelo de dominio del sistema.

Hibernate es el ORM que mapea todas las entidades y sus atributos a tablas de base de datos, relacionándolas de acuerdo a las anotaciones que vaya leyendo como `@ManyToOne` o `@OneToMany`.

Ejemplo de Repositorio

```
@Repository
public interface ImagenRepository extends JpaRepository<Imagen, String> {

    @Query("SELECT i FROM Imagen i WHERE i.id = :id AND i.eliminado = false")
    Optional<Imagen> findActive(@Param("id") String id);

    Optional<Imagen> findByIdAndEliminadoFalse(String id);

    List<Imagen> findByTipoImagenAndEliminadoFalse(TipoImagen tipoImagen);

    List<Imagen> findByEliminadoFalse();
}
```

Ejemplo de Entidad mapeada por Hibernate

Ingeniería del Software II

```
@Entity
@Table(name = "cliente")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@SuperBuilder
@ToString(callSuper = true, exclude = "nacionalidad")
public class Cliente extends Persona {

    @ManyToOne
    @JoinColumn(name = "nacionalidad_id", nullable = false)
    private Nacionalidad nacionalidad;
}
```

DTO

Se encuentra principalmente aplicado en las clases del paquete reporte dentro del paquete dto.

```
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class ReporteVentasDTO {
    private LocalDate fechaDesde;
    private LocalDate fechaHasta;
    private double totalRecaudado;
    private int cantidadVentas;
    private int totalArticulosVendidos;
    private double ticketPromedio;

    @Builder.Default
    private List<ReporteVentaItemDTO> items = new ArrayList<>();
}
```

Ingeniería del Software II

Agrupar datos de fechas, cantidades y una lista de items para transportarlos a la vista AdminReportController y mostrarlos sin exponer atributos sensibles de las clases de Producto, Factura, Cliente, Detalle y VigenciaPrecio.

El funcionamiento de demás DTOs para los reportes tienen la misma aplicación y finalidad del patrón.

Inyección de Dependencias

Es un patrón de diseño donde un objeto no crea sus propias dependencias (los recursos u otros objetos que necesita para funcionar), sino que las recibe desde el exterior. Esto busca desacoplar el código entre clases para aumentar la mantenibilidad, reutilización y flexibilidad del software.

Prácticamente todas las clases del sistema Zero cumplen con éste patrón. Se decidió que la inyección sea por constructor porque ésto nos permite declarar la inmutabilidad de los objetos que son inyectados y garantizar un estado válido desde el inicio dado que el objeto no va a poder ser instanciado si no se le proporcionan sus dependencias.

Ejemplo de aplicación

```
@Service
public class StockService {

    private final StockRepository stockRepository;
    private final DetalleRepository detalleRepository;

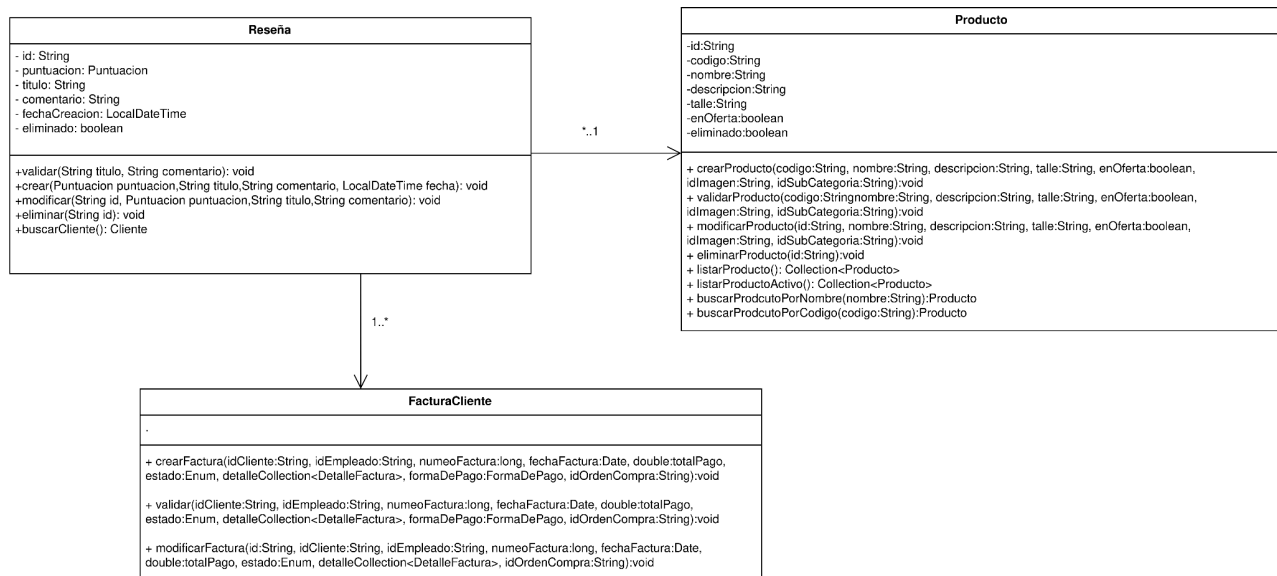
    public StockService(StockRepository stockRepository, DetalleRepository detalleRepository) {
        this.stockRepository = stockRepository;
        this.detalleRepository = detalleRepository;
    }
}
```

Ingeniería del Software II

4. Propuesta de mejora

El equipo propone la funcionalidad de reseñas a productos. Dichas reseñas contarán con puntuación del 1 al 5, título, comentario, fecha de creación y eliminado para el borrado lógico.

Una reseña tendrá un cliente que la hace y un producto al que se refiere. Además se debe poder verificar que el producto reseñado fue realmente comprado por ese cliente, por lo que debe estar asociado a una factura. Desde factura puede accederse al cliente y verificarse que el producto fue comprado.





Ingeniería del Software II

5. Tecnologías

1. Backend y Frameworks

- **Java 25**
- **Spring Boot**
- **Spring MVC**
- **Spring Data JPA y Hibernate** (ORM) y repositorios para consultas y persistencia.
- **Spring Security**: Control de autenticación y autorización, encriptación de credenciales (BCrypt) y gestión de roles (ADMIN, CLIENTE).
- **Spring Mail (JavaMailSender)**
- **Spring Scheduling**: Ejecución programada en segundo plano de tareas automatizadas (newsletter y actualización de precios por inflación).
- **Lombok**: Generación automática de código repetitivo (getters, setters, constructores, patrones builder).

2. Base de Datos

- **MySQL**
- **H2 Database**: Base de datos en memoria utilizada para aislar y acelerar la ejecución de tests automatizados.

3. Integraciones & Servicios Externos

- **Mercado Pago Java SDK**: Pasarela de pago para el checkout de la tienda (creación de preferencias de pago y recepción de webhooks de confirmación).
- **Apache POI (OOXML)**: Generación y exportación de reportes administrativos en planillas Excel (.xlsx).
- **SMTP Gmail**: Servidor de correo saliente configurado para la entrega de emails.



Ingeniería del Software II

4. Frontend

- **Thymeleaf:** Motor de plantillas del lado del servidor para renderizar HTML dinámico, fragmentos y control condicional de vistas según roles.
- **HTML5 & CSS3**
- **JavaScript (Vanilla y jQuery):** Lógica del cliente, manejo interactivo del carrito de compras, validaciones en formularios.
- **Bootstrap (v4/v5) y Iconografías (Bootstrap Icons, FontAwesome, Themify)**

5. Testing

- **JUnit:** Pruebas unitarias.
- **Selenium:** Pruebas de integración End-to-End (E2E) automatizadas en navegador real (flujos de login, compras a proveedores y ventas con control de stock).
- **JMeter:** Pruebas de carga y estrés con requests GET y POST a distintas urls.

6. Requerimientos no funcionales

Seguridad (Security)

1. Cifrado y hashing de contraseñas (Confidencialidad)

- **Descripción:** Se utiliza el algoritmo de derivación de claves BCrypt con salt automático (evita el caso de hash idéntico frente a claves repetidas).
- **Implementación:**

Ingeniería del Software II

```
SecurityConfig.java:
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

UsuarioService.java:
Usuario usuario = Usuario.builder()
    .nombreUsuario(usuarioLimpio)
    .clave(passwordEncoder.encode(clave))
    .rol(rol)
```

2. Control de acceso y autorización basada en roles (RBAC)

- **Descripción:** Se protege el acceso al panel administrativo verificando la sesión activa y los privilegios de usuario.
- **Implementación:**

```
AdminAuthInterceptor.java:
java
@Override
public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) throws Exception {
    HttpSession session = request.getSession(false);
    Usuario usuario = (session != null) ? (Usuario) session.getAttribute("usuariosession") : null;
    if (usuario == null) {
        response.sendRedirect("/admin/login");
        return false;
    }
    if (usuario.getRol() != RolUsuario.ADMINISTRATIVO && usuario.getRol() != RolUsuario JEFE) {
        response.sendRedirect("/?error=unauthorized");
        return false;
    }
    return true;
}
```

Ingeniería del Software II

3. Seguridad de autenticación en dos factores / Códigos con caducidad temporal

- **Descripción:** Los códigos de confirmación por email expiran transcurrido su periodo de validez para mitigar ataques de fuerza bruta o secuestro de cuentas.
- **Implementación:**

```
UsuarioService.java:
java
if (usuario.getCodigoConfirmacion() == null || usuario.getCodigoExpiracion() == null) {
    throw new IllegalArgumentException("No hay ningún código de confirmación pendiente. Solicita uno nuevo.");
}
if (LocalDateTime.now().isAfter(usuario.getCodigoExpiracion())) {
    throw new IllegalArgumentException("El código de confirmación ha expirado. Por favor solicita uno nuevo.");
}
```

Integridad de Datos y Concurrencia (Data Integrity & Consistency)

1. Atomicidad y transaccionalidad (ACID)

- **Descripción:** Las operaciones que involucran múltiples cambios (descuento de inventario, persistencia de compras, facturación y cobro) se ejecutan dentro de transacciones atómicas. Si una falla, se revierte todo el lote (rollback).
- **Implementación:**

Ingeniería del Software II

```
ProductoService.java:
java
@Transactional
public void eliminarProducto(String id) {
    Producto producto = productoRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("No se encontró el producto con id: " + id));
    producto.setEliminado(true);
    productoRepository.save(producto);
}
```

2. Borrado Lógico (Soft Delete)

- **Descripción:** Se previene la destrucción accidental o física de registros preservando la integridad referencial histórica mediante el boolean eliminado.

Rendimiento y Eficiencia de Recursos (Performance & Resource Management)

1. Paginación y procesamiento en lotes (Batch Processing)

- **Descripción:** Para evitar sobrecarga y errores de falta de memoria (OutOfMemoryError), operaciones masivas (como el reajuste bimestral de precios) se procesan paginadas por lotes.
- **Implementación:**

Ingeniería del Software II

ProductoService.java:

```
do {  
    Pageable pageable = PageRequest.of(pageNumber, tamanoPagina, Sort.by("id").ascending());  
    pagina = productoRepository.findByEliminadoFalse(pageable);  
    List<Producto> lote = (pagina != null) ? pagina.getContent() : List.of();  
    int totalPaginas = (pagina != null && pagina.getTotalPages() > 0) ? pagina.getTotalPages() : 1;  
    logger.info("Procesando lote de precios - Página {} de {} ({} productos)",  
        pageNumber + 1, totalPaginas, lote.size());  
}
```

2. Carga perezosa (Lazy Loading) en mapeos ORM

- **Descripción:** Las relaciones @ManyToOne, @OneToMany y @OneToOne se configuran con FetchType.LAZY para evitar traer árboles completos de objetos innecesarios a memoria.
- **Implementación:**

Producto.java:

```
java  
@OneToMany(fetch = FetchType.LAZY)  
private List<Imagen> imagenes;  
@ManyToOne(fetch = FetchType.LAZY)  
private SubCategoria subCategoria;
```

Ingeniería del Software II

3.Optimización de subida de archivos (Protección contra Denial of Service)

- **Descripción:** Límites explícitos en el tamaño de subidas de archivos tanto a nivel de contenedor Tomcat como en Spring MVC.
- **Implementación:**

```
application.properties:
properties
spring.servlet.multipart.enabled=true
spring.servlet.multipart.max-file-size=50MB
spring.servlet.multipart.max-request-size=50MB
server.tomcat.max-swallow-size=50MB
server.tomcat.max-http-form-post-size=50MB
```

Tolerancia a Fallos (Fault Tolerance & Reliability)

1. Manejo centralizado de excepciones globales

- **Descripción:** Se interceptan fallos no controlados y desbordes de payload sin exponer stack traces ni detalles técnicos al usuario final.
- **Implementación:**

Ingeniería del Software II

```
GlobalExceptionHandler.java:
java
@ExceptionHandler({MaxUploadSizeExceededException.class, MultipartException.class})
public String handleMaxSizeException(Exception exc, HttpServletRequest request, RedirectAttributes redirectAttributes) {
    redirectAttributes.addFlashAttribute("errorMessage", "El archivo seleccionado es demasiado grande.");
    String referer = request.getHeader("Referer");
    if (referer != null && !referer.isBlank()) {
        return "redirect:" + referer;
    }
    return "redirect:/register";
}
```

2. Recuperación controlada de errores HTTP (404/500 Friendly Error Pages)

- **Descripción:** Enrutamiento de peticiones erróneas o recursos inexistentes hacia vistas controladas.
- **Implementación:**

```
CustomErrorController.java:
java
if (statusCode == HttpStatus.NOT_FOUND.value()) {
    model.addAttribute("errorMessage", "La página o recurso que buscas no existe o ha sido movido temporalmente.");
    return "admin/page-404";
}
```

Ingeniería del Software II

Automatización y Operabilidad (Operability & Scheduling)

1. Ejecución desatendida de tareas periódicas (Cron Jobs en segundo plano)

- **Descripción:** Los procesos repetitivos (actualización de precios e índices de inflación, envíos de newsletters) operan de forma automatizada sin intervención manual.
- **Implementación:**

```
NewsletterService.java:
java
@Scheduled(cron = "0 0 9 */10 * *")
public void enviarNewsletterOfertasProgramado() {
    logger.info("Iniciando envío programado de newsletter de ofertas...");
    enviarNewsletterOfertas();
}

ProductoService.java:
java
@Scheduled(cron = "${inflacion.cron:0 0 2 1 */2 ?}")
public void actualizarPreciosPorInflacionProgramado() {
    logger.info("Ejecutando tarea programada: Actualización de precios por inflación...");
}
```

Mantenibilidad y Trazabilidad (Maintainability & Observability)

1. Registro estructurado de eventos y auditoría (Logging)

- **Descripción:** Implementación de Logback para dejar traza de eventos de negocio, fallos y ejecuciones críticas.
- **Implementación:**

Ingeniería del Software II

```
EmailService.java:
java
private static final Logger logger = LoggerFactory.getLogger(EmailService.class);
// ...
try {
    mailSender.send(message);
    logger.info("Correo enviado exitosamente a: {}", to);
} catch (Exception e) {
    logger.error("Error al enviar correo a {}: {}", to, e.getMessage(), e);
    throw new RuntimeException("No se pudo enviar el correo de verificación.", e);
}
```

2.Arquitectura en Capas y Verificación Automatizada (Architecture Governance)

- **Descripción:** Separación clara de responsabilidades (Controladores, Servicios, Repositorios, Entidades, DTOs) protegida mediante pruebas de arquitectura automatizadas.
- **Implementación:** Una clase que valida que las capas superiores no violen el desacoplamiento ni salten responsabilidades entre paquetes.